

Qubit Parameter Identification

Sergei Leonov

(Dated: March 14, 2026)

This paper describes the procedure of identifying the unknown qubit characteristics using simulated environment Qublitz. The author employs Lorentzian model to fit the frequency sweep data to extract qubit frequency. Rabi model is used to analyze the oscillation rate when qubit is driven on resonance. Finally, qubit's relaxation time is estimated by exciting qubit with a π -pulse and measuring time constant of the first order fit.

PACS numbers:

INTRODUCTION

In quantum engineering, the problem of understanding the properties of the quantum bit (qubit) are central to improving the efficiency of a quantum computer. Researchers developed many sophisticated ways to construct two-level-systems (TLS) that provide viable options for preserving a qubit [1–3].

A property that dramatically affects qubit's ability to store information is its coherence time (T_1). It encodes the time constant of qubit's decay from *excited state* $|1\rangle$ to *ground state* $|0\rangle$. One of the main lines of quantum engineering research is aimed at maximizing T_1 using control signals, more efficient quantum devices, or new materials [1, 3–5].

It is often of interest how qubit behaves under varying control pulses and how those affect its state. Control techniques have been developed for manipulating qubits to enhance coherence time [1]. In order to efficiently construct control pulses, one needs to know the qubit frequency ω_q because resonance occurs when qubit is driven with frequency $\omega_d \approx \omega_q$. This resonant drive results in the largest amplitude of the response.

Another important quantity is the Rabi frequency ω_{rabi} or Ω . It specifies the rate of Rabi oscillations of the qubit between $|1\rangle$ and $|0\rangle$ when driven at resonant frequency. This becomes useful when constructing control pulses to perform quantum gates such as the X -gate. Knowing the Rabi rate allows to reduce error accumulation during the sequential applications of logical gates to the qubit.

That said, extracting these parameters effectively is central to understanding how a given qubit behaves. This manuscript outlines a series of experiments aimed at identifying the above mentioned properties from an unknown qubit. The experiments are performed in the simulated *Qublitz* environment that was developed by researchers at FitzLab. The assumptions for the analysis that follows are:

1. Qubit obeys the general driven time-dependent Hamiltonian:

$$\hat{H}/\hbar = \frac{\omega_q}{2}\hat{\sigma}_z + \frac{\Omega(t)}{2}\hat{\sigma}_x \cos(\omega_d t) + \frac{\Omega(t)}{2}\hat{\sigma}_y \cos(\omega_d t) \quad (1)$$

2. Dephasing time T_2 is exactly double the decoherence time T_1 :

$$T_2 = 2T_1 \quad (2)$$

EXPERIMENTS AND RESULTS

This section provides a summary of the experiments performed to collect the data that is later analyzed by fitting models to estimate the quantities of interest. For analysis, MATLAB's *Curve Fitting Toolbox* was employed to fit the model parameters to data. Covariance matrices were extracted from the fit Jacobian matrix (J) and Mean Square Error (MSE) using the formula $MSE \cdot (J^T J)^{-1}$. The results of the parameter fitting along with uncertainties are summarized in table I.

Determining the Qubit Frequency ω_q

Using Qublitz, the maximum probability of excited state was estimated for various drive frequency (ω_d) values from 4.70 GHz to 5.20 GHz with pulse duration of 25 ns. The interval included 201 data points. The Lorentzian model (eq. 3) was used to estimate the qubit frequency ω_q ,

$$P_{\text{max}}(\omega_d) = A \frac{(\Gamma/2)^2}{(\omega_d - \omega_q)^2 + (\Gamma/2)^2} + C \quad (3)$$

where A is the amplitude, Γ is linewidth, and C is vertical offset.

The resulting fit of $P_{\text{max}}(\omega_d)$ against a range of drive frequencies is shown in fig. 1. The root mean squared error (RMSE) for the fit was 0.0041, indicating a well-fitted model. Extracted parameter ω_q was estimated to be 4.9521 GHz with uncertainty of 0.1 MHz, which agrees with the estimate provided by Qublitz software.

Determining the Rabi Rate ω_{rabi}

To measure the Rabi rate, qubit is driven at resonant frequency $\omega_d = \omega_q$, where ω_q was previously estimated

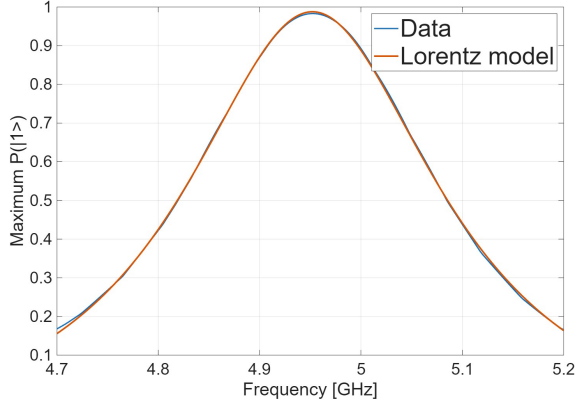


FIG. 1: Frequency sweep data with Lorentzian fit for $P_{\max}(|1\rangle)$

TABLE I: A summary of the extracted parameters from fitted models along with the corresponding uncertainties derived from the covariance matrix of each fit

Parameter	Value	Units
ω_q	4.9521 ± 0.0001	GHz
ω_{rabi}	51.4094 ± 0.0348	MHz
T_1	54.5412 ± 0.1501	ns

at 4.9521 GHz. The drive pulse had an amplitude of 1.0, and the oscillation of probability of excited state $|1\rangle$ was observed for 30 ns. The resulting data was fitted against the Rabi oscillation model (eq. 4),

$$P_{|1\rangle}(t) = A \sin^2(4\pi \omega_{\text{rabi}} t) e^{-t/\tilde{T}_1} + C \quad (4)$$

where A is the amplitude, ω_{rabi} is Rabi rate (in GHz), $\tilde{T}_1$ is driven decoherence rate, and C is vertical offset. Note that a factor of 4π appears because:

$$\begin{aligned} T_{\text{osc}} &= \frac{2\pi}{\omega_{\text{osc}}} \quad \text{and} \quad \omega_{\text{rabi}} = \frac{1}{2T_{\text{osc}}} \\ \Rightarrow T_{\text{osc}} &= \frac{1}{2\omega_{\text{rabi}}} = \frac{2\pi}{\omega_{\text{osc}}} \\ \Rightarrow \omega_{\text{osc}} &= 4\pi\omega_{\text{rabi}} \end{aligned}$$

The fit and data is displayed in fig. 2. The time was scaled to microseconds to obtain better uncertainty estimates. The RMSE for the fit is 0.1250, which is significant error. This is easy to see from the graph as the sinusoid models the data well at the peaks, but not at the valleys. This discrepancy at the valleys of the plot, results in high RMSE value. However, the model still accurately captures the period of oscillation as the centers of all peaks and valleys of the fit and data align throughout the recorded time frame. Therefore, the Rabi rate is well-predicted by the fit at 51.4094 ± 0.0348 MHz.

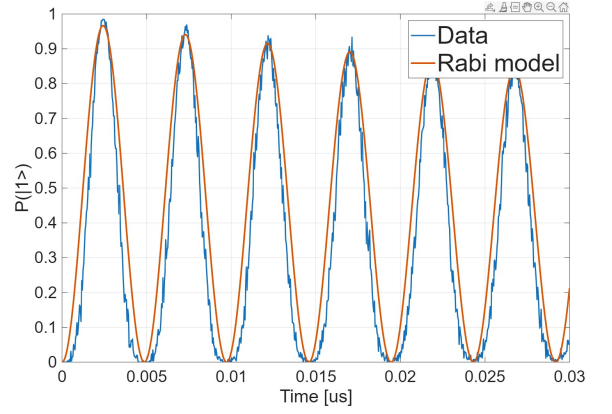


FIG. 2: Rabi model fit to oscillatory $P_{|1\rangle}$ evolution at resonant drive

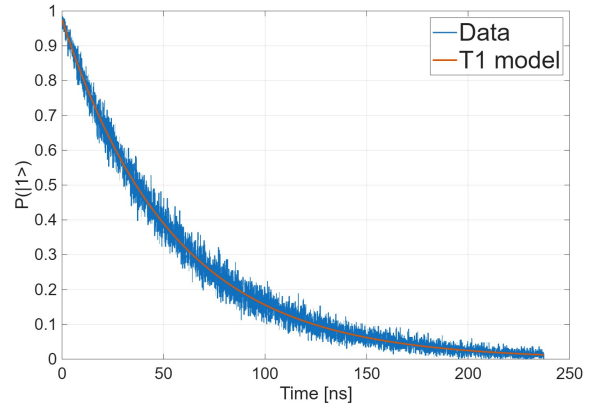


FIG. 3: $P(|1\rangle)$ decay with first order fit after π -pulse

Determining the Relaxation Time T_1

For the final experiment, a π -pulse at the resonant frequency $\omega_d = \omega_q$ was applied to a qubit in the ground state $|0\rangle$ to excite it to $|1\rangle$. The pulse duration was 3 ns with amplitude 0.85. The data was collected for 240 ns. Once the drive pulse was turned off, the qubit exhibited exponential decay with characteristic time T_1 . Figure 3 demonstrates the resulting data with fitted first order model described in eq. (5). A π -pulse period was cropped for easier RMSE analysis.

$$P_{|1\rangle}(t) = P_0 e^{-t/T_1} + C \quad (5)$$

The RMSE for the plot in fig. 3 was 0.0207, indicating a good model fitness. The extracted time of qubit relaxation was found to be 54.5412 ± 0.1501 ns. Therefore, after $5T_1 = 272.706 \pm 0.7505$ ns, the qubit loses all its energy and drops back to the ground state.

CONCLUSION

The above experiments provide valuable insight into the structure of the unknown qubit. Lorentzian, Rabi, and first order models serve as good approximations of qubit behavior and prove useful in measuring the unknown quantities of interest. It is of future interest to assess the fitness of these models to the data extracted from the evolution of real qubits prepared in a quantum computer. Also, sophisticated noise removal techniques could be explored to enhance model fitting. Figures 2 and 3 demonstrate that $P_{|1\rangle}$ data becomes noisy due to stochasticity. It would also be interesting to explore the nature of the disagreement of the Rabi model with the experimental data at the valleys of oscillatory response. All in all, the outlined experiments provide a sufficient baseline estimate for the qubit parameters. However, more advanced models are needed to make more precise claims about the qubit at hand.

[1] Coherent Manipulation of Coupled Electron Spins in Semiconductor Quantum Dots. J. R. Petta. Science 309

(5744). DOI: 10.1126/science.1116955

[2] Daniel Gottesman, Alexei Kitaev, and John Preskill. Encoding a qubit in an oscillator. Phys. Rev. A, 64(1):012310, 2001

[3] Long B. Nguyen, Yen-Hsiang Lin, Aaron Somoroff, Raymond Mencia, Nicholas Grabon, and Vladimir E. Manucharyan. High-coherence fluxonium qubit

[4] A. D. O’Connell, M. Hofheinz, M. Ansmann, R. C. Bialczak, M. Lenander, E. Lucero, M. Neeley, D. Sank, H. Wang, M. Weides, J. Wenner, J. M. Martinis, and A. N. Cleland. Quantum ground state and single-phonon control of a mechanical resonator. Nature, 464:697–703, 2010.

[5] Matthew P. Bland, Faranak Bahrami, Jeronimo G. C. Martinez, Paal H. Presteggaard, Basil M. Smitham, Atharv Joshi, Elizabeth Hedrick, Shashwat Kumar, Ambrose Yang, Alexander C. Pakpour-Tabrizi, Apoorv Jindal, Ray D. Chang, Guangming Cheng, Nan Yao, Robert J. Cava, Nathalie P. De Leon, and Andrew A. Houck. Millisecond lifetimes and coherence times in 2d transmon qubits. Nature, 2025

```

%% ENGS 53; Extracting Qubit Frequency
%
% 12/03/2026 Sergei Leonov

close all
clear
clc

data = readtable( frequency_sweep_summary.csv );
freq = data.omega_d_GHz;
prob = data.max_prob_1;

% define the model
lorentz = fittype('A * ((G/2)^2 ./ ((x - wq).^2 + (G/2)^2)) + C', ...
    'independent', 'x', 'coefficients', {'A', 'G', 'wq', 'C'
    });

% initial guess
A = 1;
G = 0.5;
wq = 5;
C = min(prob);
b = [A, G, wq, C];

% fit the model
[f, gof, out] = fit(freq, prob, lorentz, 'StartPoint', b)

% compute the fit
A = f.A
G = f.G
omega_q = f.wq
C = f.C

y = A * ((G/2)^2) ./ ((freq - omega_q).^2 + (G/2)^2) + C;

% covariance matrix
J = out.Jacobian;
mse = gof.rmse^2;
cov_mat = mse*inv(J'*J);
sqrt(diag(cov_mat))

rmse(prob, y)

% plot data and model
fig = figure;
plot(freq, prob, '-', 'LineWidth', 1.6);
grid on; hold on;
plot(freq, y, '-', 'LineWidth', 2);
legend( Data , Lorentz model , FontSize , 30);
xlabel( Frequency [GHz] , FontSize , 20);
ylabel( Maximum P(|1>) , FontSize , 20);
set(gca, 'FontSize', 20);
saveas(fig, freq.jpeg )

```

```

%% ENGS 53; Extracting Rabi Rate
%
% 12/03/2026 Sergei Leonov

close all
clear
clc

data = readtable( time_domain_sim.csv );
time = data.time_ns*10^(-3); % convert to microseconds
prob = data.p1_meas;

% define model parameters
A = 1;
omega = 50; % [MHz]
T1 = 0.180; % [us]
C = 0;

% define new model
sin2_fit = fittype('A * sin(4*pi* B * x)^2 * exp(-x/T1)', ...
    'independent', 'x', 'coefficients', {'A', 'T1', 'B'});
start_pts = [A, T1, omega];

% fit the model
[f, gof, out] = fit(time, prob, sin2_fit, 'StartPoint', start_pts, Lower,
    [0.98 0.180, 0])

% compute model
A = f.A
T1 = f.T1
omega = f.B

y = A*(sin(4*pi*omega*time)).^2.*exp(-time/T1);

% compute covariance matrix
J = out.Jacobian;
mse = gof.rmse^2;
cov_mat = mse*inv(J'*J);
sqrt(diag(cov_mat))

% plot data and model
fig = figure;
plot(time, prob, '-', 'LineWidth', 1.6);
grid on; hold on;
plot(time, y, '-', 'LineWidth', 2);
legend( Data, Rabi model, FontSize, 30);
xlabel( Time [us], FontSize, 20);
ylabel( P(|1>), FontSize, 20);
set(gca, 'FontSize', 20);
saveas(fig, Rabi.jpeg)

```

```

%% ENGS 53; Extracting T1
%
% 12/03/2026 Sergei Leonov

close all
clear
clc

data = readtable( time_domain_T1_data.csv );

time = data.time_ns; % nanoseconds
prob = data.p1_meas;

% crop data to extract transient
start = find(prob >= 0.98, 1, 'first');
time = time(start:end); % crop time
time = time - time(1); % start time at zero
prob = prob(start:end); % crop prob

% define the model
exp_model = fittype('A * exp(-x/T1) + C', ...
    'independent', 'x', 'coefficients', {'A', 'T1', 'C'});

% define initial guess parameters
A = 1;
T1 = 50;
C = 0;

% initial guess
b = [A T1 C];

% extract model parameters
[f, gof, out] = fit(time, prob, exp_model, StartPoint, b)
A = f.A
T1 = f.T1
C = f.C

% compute predicted model
y = A*exp(-time/T1) + C;

% compute covariance matrix
J = out.Jacobian;
mse = gof.rmse^2;
cov_mat = mse*inv(J'*J);
sqrt(diag(cov_mat))

% plot data and model
fig = figure;
plot(time, prob, '-', 'LineWidth', 1.6);
grid on; hold on;
plot(time, y, '-', 'LineWidth', 2);
legend( Data, T1 model, FontSize, 30);
xlabel( Time [ns], FontSize, 20);
ylabel( P(|1>), FontSize, 20);
set(gca, 'FontSize', 20);
saveas(fig, T1.jpeg)

```